Make a PDF file with your name <FirstName_LastName>. The file should have the following information, and must be updated after every presentation.

Presentation 1

- Links to presentation(s) and code(s) on GitHub
    - https://github.com/arvindkrishna87/STAT390_LegalAid_Fall2025/blob/main/Presentations/Menu%20redundancy%20analysis/RedundantPaths_09Oct_VincentWillYiHyun.pdf
- What did you do?
    - Along with my teammates Will and Yihyun, we defined what redundancies are, proposed 3 types of redundancies, as well as proposed algorithms for finding these redundancies. I worked primarily on the first type of redundancy, "Calls taking longer than they should". I thought of two subtypes of the redundancy and proposed a data-based algorithm for identifying these redundancies in the data, as well as identifying the key challenges that the algorithms would have to address to help guide next steps.
- How does it help the project?
    - My work helps the project by clearly defining what redundancies are and by proposing algorithms to detect them, specifically for cases where calls take longer than expected. This creates a concrete framework for identifying the problem that our client is facing through the data and guides the next phase of analysis and solution development.
- Issues faced (if any): N/A
- Attempts to resolve issues (if any): N/A
- Issues resolved (if any): N/A
- Next steps
    - There are a few tasks I hope to work on. Firstly, I would like to deep dive into the redundancy of "Calls taking longer than they should".
        - For the first subtype, I would like to carry out the algorithm and calculate idle time accurately adjusting for other factors. A good first step would be calling and recording the time it took for each prompt. I am not entirely sure how to adjust for factors other than prompt length (ex: time of the day).

        - For the second subtype, I would like to start by just identifying a few (maybe only one or two) instances of important final nodes. I would like to implement the algorithm where we find the set of all paths and the average time taken by each path to reach that final node and output those paths that seem to take the most time.
    - Additionally, I would also like to help implement the algorithms needed to identify repeated processes (second type of redundancy). Additionally, I would like to think of the kind of data structure we should use to represent our data, especially since our algorithm for identifying repeated processes relies on the notion of parent-child nodes, as well as cycles.
    - The above goals I've mentioned comprise of a lot of content. Since quality typically beats quantity, it may be beneficial for me/my team to focus deeply on only one of the goals.
- References (Mention if you built up on someone else's work): We referenced an academic paper that suggested that people can only retain 3-5 meaningful things in their minds at once. The citation is as below:
    - Cowan, N. (2010). *How is Working Memory Capacity Limited, and Why?* National Library of Medicine (PMC). https://pmc.ncbi.nlm.nih.gov/articles/PMC2864034/

Presentation 2

- Links to presentation(s) and code(s) on GitHub
    - https://github.com/arvindkrishna87/STAT390_LegalAid_Fall2025/blob/main/Presentations/Menu%20redundancy%20analysis/RepeatedProcesses_23Oct_WillVinceYiHyun.pdf
    - My code: https://github.com/arvindkrishna87/STAT390_LegalAid_Fall2025/blob/main/Code/Menu%20redundancy%20analysis/PromptRepetitions_23Oct_Vincent.ipynb
- What did you do?
    - Along with my teammates, I brainstormed how we would work with the data in the Legal Menu and Main Menu, additional columns that might be useful to include, and a preliminary definition of success.

    - Also, I looked specifically into the cases where a caller listened to the full prompt but still needed to repeat hearing the prompt. I analyzed the prompts (7 for now) which had the most repetitions as well as redundancy ratio to identify the prompts that are causing the most node-level confusion.
- How does it help the project?
    - It helps us identify (at a basic level) which prompts are causing node-level confusion, in order for us to understand how to change the verbiage.
- Issues faced (if any)
    - The inability to distinguish between Spanish and English prompts. This is a problem with precisely identifying what callers actually hear the full prompt or not.
- Attempts to resolve issues (if any)
    - We were not able to find any way in CAR data to help us explicitly determine whether a call was in Spanish or English.
- Issues resolved (if any)
    - None
- Next steps
    - I'd like to determine whether it's possible to identify if a prompt is in Spanish or English.
    - If language identification is possible, we could precisely measure the redundant time associated with each prompt and calculate exactly how many/which options a caller hears before selecting one. This would allow us to better understand caller behavior and confusion patterns, and to pinpoint when callers begin recognizing the correct option, which could inform a more effective prompt ordering.
    - If language identification is not possible, then node-level analysis would be significantly limited. In that case, I would shift focus toward analyzing call-level redundancies and repeated call paths instead, which is what Will and Yihyun have primarily focused on. I will extend their analyses based on what we decide to work on after our TA meeting on Friday.
- References (Mention if you built up on someone else's work)

Presentation 3

- Links to presentation(s) and code(s) on GitHub
    - https://github.com/arvindkrishna87/STAT390_LegalAid_Fall2025/blob/main/Presentations/Menu%20redundancy%20analysis/MenuRepetitionsandSuccessfulCallAnalysis_4Nov_WillVinceYiHyun.pdf
    - https://github.com/arvindkrishna87/STAT390_LegalAid_Fall2025/blob/main/Code/Menu%20redundancy%20analysis/RepeatsSuccessAnalysis_4Nov_Vincent.ipynb
- What did you do?
    - With my teammates, we thought of the next steps for our analysis, and I was in charge of exploring whether repetitions can help a user reach a queue or not.
    - To proceed with the analysis, I added a set of additional columns, including whether a call is successful, total number of repeats per call, total number of repeats per prompt in a given call, total number of movements (change in node), number of unique nodes for a given call, as well as the path (represented by a list, or a string joined by "->" for each given call. These new columns will be important for this analysis, as well as our analyses moving forwards, as they directly represent that call level behavior of users. I then analyzed whether more repeats were associated with more successful calls or less successful calls. I did this for the full call, and for specific repetitions in Legal and Main Menu.
    - I also proposed looking at a new structure, path, that provides a granular way that essentially captures the entire information of a call (except for time).
- How does it help the project?
    - It helps us understand whether repetitions are helpful for the user. It guides our next step towards understanding how might repetitions help vs how might repetitions harm. We now know that a small number of repetitions may be helpful for the user, but these are not exactly Legal Menu or Main Menu. When we find the exact prompts that are helpful, we may be able to design the menu more effectively to optimize prompts and repetitions by introducing the helpful prompts more.
- Issues faced (if any)
- Attempts to resolve issues (if any)
- Issues resolved (if any)

    - None
- Next steps
    - Find out exactly which prompts are the ones that are the ones that may be helpful when repeated.
    - Analyze the paths of calls, and what patterns show in calls that are successful vs calls that aren't. I would like to start implementing the algorithm in presentation 1 (which we haven't used yet), where for each given final node, we look at the set of paths that may lead to the final node and see which of the paths may be redundant.

References (Mention if you built up on someone else's work)